

Chain Poker Genesis

Table Permission Widget Specification

1. Widget Purpose

The **Table Permission Widget** is the visible interface of the **Request and Permission Engine** within Chain Poker Genesis.

Its purpose is not to decide the game by itself, but to show the player which actions are permitted, which actions are not permitted, and the reason according to the verified real-time state of the table.

The widget exists to:

- make the state machine visible;
 - prevent invalid action requests;
 - block inconsistent intentions before they become irreversible actions;
 - show the player the exact logic of their current situation;
 - provide an early detection mechanism for possible conflicts.
-

2. Design Principle

The widget is not the source of truth.

The source of truth is always:

- Genesis State Machine;
- Poker Rules Engine;
- Position Engine;
- Request and Permission Engine;
- verified table state.

The widget only reflects the result generated by these components.

The widget does not create permissions. It displays permissions already calculated by the protocol.

3. Specific Function

The widget must answer questions such as:

- Can I call?
- Can I raise?
- Can I go all-in?
- Can I fold?
- Why is an action not allowed?
- What valid action is available instead?
- Does the displayed state match the verified protocol state?

The widget must not only say "yes" or "no". It must also explain the reason for the restriction.

4. Example: All-In and Intent Blocking

Scenario

Player A acts first and goes **all-in**.

The next player receives the turn.

The next player attempts to request an action:

- call;
- raise;
- all-in.

The Request and Permission Engine evaluates whether each action is valid in the current state.

Widget Output Example

PLAYER ACTION STATUS

CALL

 Not permitted

Reason:

The required amount cannot be covered under the current condition.

ALL-IN

 Permitted

Reason:

This is the valid action available with the player's current balance.

RAISE

 Not permitted

Reason:

Insufficient funds to satisfy the minimum raise requirement.

The widget prevents the player from attempting an invalid intention.

5. Relationship Between Requested Action and Permitted Action

An action is not validated only by its name.

The protocol evaluates the relationship between:

- player intention;
- available balance;
- current bet;
- player position;
- active rules;
- current table state.

Therefore, a player may intend to make a call, but the protocol may respond that **call is not permitted** and that **all-in is the valid action**.

The widget distinguishes between:

- what the player wants to do;
 - what the protocol allows the player to do.
-

6. Intent Blocking Function

The widget acts as an **intent blocking layer**.

The player should not proceed blindly toward an invalid action.

Correct flow:

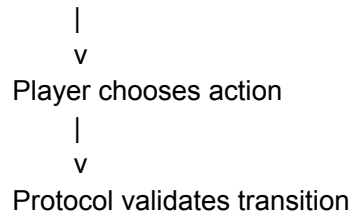
State Machine updates table state

|
v

Permission Engine calculates valid actions

|
v

Widget displays allowed and denied actions



This reduces ambiguity and unnecessary conflicts.

7. Conflict Detection

The widget must also detect when the displayed information does not match the verified state.

Example:

STATE MISMATCH

The visible information does not match the verified protocol state.

Actions temporarily blocked.

Conflict resolution may be requested.

This allows detection of:

- synchronization errors;
 - invalid client states;
 - manipulation attempts;
 - inconsistent information.
-

8. Relationship With Conflict Resolution

If the player detects that the widget is inconsistent with the expected rules, the widget may initiate a conflict resolution request.

The widget does not resolve the conflict.

It only creates the evidence point for escalation.

Process:

Widget displays verified permission state

|

v

Player detects inconsistency

|

v

Conflict request created

|

v

Current state is preserved

|

v

Consensus verifies the state

9. What the Widget Must and Must Not Do

The widget must:

- display current permissions;
- display blocked actions;
- explain restrictions;
- show valid alternatives;
- reveal inconsistencies;
- provide a transparent view of the state machine.

The widget must not:

- determine the winner;
- modify poker rules;
- alter protocol state;
- create permissions;
- replace consensus;
- hide inconsistencies.

10. Architectural Value

The Table Permission Widget converts internal protocol logic into a visible and understandable interface for players.

Benefits:

- transparency;
- immediate validation;

- fewer intention errors;
- reduced operational ambiguity;
- improved auditing;
- clearer verification of machine behavior.

In Chain Poker Genesis, this is important because the protocol relies on **verifiable states rather than trust between participants**.

11. Conclusion

The Table Permission Widget is a critical component because it shows the player what actions are and are not possible in the current verified state.

In an all-in scenario, the widget can inform the player:

- call is not permitted;
- all-in is permitted;
- raise is not permitted due to insufficient funds;
- the table state is inconsistent;
- conflict resolution should be requested.

Its final purpose is:

To transform the state machine into a visible, verifiable, and auditable guide for the player.

Chain Poker Genesis

Especificación del Widget de Permisos de Mesa

1. Propósito del widget

El **Widget de Permisos de Mesa** es la interfaz visible del **Motor de Solicitudes y Permisos** dentro de Chain Poker Genesis.

Su función no es decidir el juego por sí mismo, sino **mostrarle al jugador qué acciones están permitidas, cuáles no lo están y por qué** según el estado verificado de la mesa en tiempo real.

Este widget existe para:

- hacer visible la máquina de estados;
 - evitar solicitudes inválidas;
 - bloquear intenciones incoherentes antes de que se conviertan en acciones irreversibles;
 - mostrar al jugador la lógica exacta de su situación actual;
 - servir como punto de detección temprana de conflictos.
-

2. Principio de diseño

El widget no es una fuente de verdad.

La fuente de verdad siempre es la combinación de:

- Máquina de Estados Génesis;
- Motor de Reglas de Poker;
- Motor de Posición;
- Motor de Solicitudes y Permisos;
- estado verificable de la mesa.

El widget solamente **refleja** el resultado de esos motores.

En otras palabras:

El widget no inventa permisos. El widget muestra permisos ya calculados por el protocolo.

3. Función específica

El widget debe responder, en tiempo real, a preguntas como:

- ¿puedo hacer call?
- ¿puedo hacer raise?
- ¿puedo hacer all-in?
- ¿puedo hacer fold?
- ¿por qué una acción no está permitida?
- ¿qué acción alternativa sí está permitida?
- ¿el estado mostrado coincide con el estado verificado por el protocolo?

Por eso el widget no solo dice “sí” o “no”. Debe comunicar también la **razón de la restricción**.

4. Ejemplo base: all-in y bloqueo de intención

Escenario

Jugador A actúa primero y hace **all-in**.

Luego le toca al siguiente jugador.

Ese siguiente jugador intenta solicitar una acción, por ejemplo:

- call;
- raise;
- all-in.

Pero el motor de permisos puede determinar que algunas de esas acciones no están permitidas en ese estado.

Ejemplo de salida del widget

ESTADO DE ACCIONES DEL JUGADOR

CALL

 No permitido

Motivo:

La apuesta requerida no puede ser cubierta bajo esta condición.

ALL-IN

 Permitido

Motivo:

Es la única solicitud válida con el saldo disponible.

RAISE

 No permitido

Motivo:

No existen fondos suficientes para cumplir el incremento mínimo.

En este caso, el widget cumple su función principal: **evitar que el jugador intente una intención inválida.**

5. Relación entre acción declarada y acción permitida

En Chain Poker Genesis, una acción no se valida solo por su nombre.

La acción se valida por la relación entre:

- la intención del jugador;
- el saldo disponible;
- la apuesta actual;
- la posición en la mano;
- las reglas vigentes;
- el estado de la mesa.

Por eso puede ocurrir que un jugador quiera hacer call, pero el protocolo responda que call **no tiene permiso** y que la acción correcta es all-in.

Eso significa que el widget ayuda a distinguir entre:

- **lo que el jugador quiso hacer;**
 - **lo que el protocolo realmente autoriza.**
-

6. Función frente al bloqueo de intención

El widget debe funcionar como una capa de **bloqueo de intención**.

Eso significa que el jugador no debería avanzar a una acción no válida sin antes ver que el sistema la rechaza.

El flujo correcto es:

1. la máquina de estados actualiza el estado de la mesa;
2. el motor de permisos calcula las acciones válidas;
3. el widget muestra las acciones permitidas y no permitidas;
4. el jugador decide;
5. si la acción solicitada no coincide con el estado, el sistema la rechaza.

Esto evita ambigüedad y reduce conflictos innecesarios.

7. Respuesta esperada del widget

El widget debe mostrar, como mínimo:

- acción disponible o no disponible;
- razón de la restricción;
- alternativa válida, si existe;
- advertencia de conflicto, si el estado no coincide.

Ejemplo de formato funcional

CALL

✗ No permitido

RAISE

✗ No permitido

ALL-IN

✓ Permitido

FOLD

✓ Permitido

Ejemplo ampliado con causa

CALL

✗ No permitido

Motivo:

Saldo insuficiente para cubrir la apuesta requerida.

Acción alternativa:

ALL-IN

8. Caso de inconsistencia

El widget también debe mostrar cuando la información local no coincide con el estado verificado.

Si el jugador ve algo que no corresponde con el estado real, el widget no debe ocultarlo ni fingir normalidad.

Debe indicar claramente una inconsistencia.

Ejemplo

ESTADO NO COINCIDE

La información visible no coincide con el estado verificado por el protocolo.

Acciones bloqueadas temporalmente.

Se recomienda solicitar resolución de conflictos.

Este comportamiento es importante porque permite detectar errores, manipulación, desincronización o intentos de engaño antes de que ocurra una decisión irreversible.

9. Relación con la resolución de conflictos

Si el jugador detecta que el widget muestra algo incoherente, el widget debe permitir activar una solicitud de revisión.

Eso no significa que el widget resuelva el conflicto.

Significa que el widget actúa como una **señal temprana de alerta** para escalar el caso al consenso.

El flujo sería:

1. el widget muestra el permiso calculado;
2. el jugador detecta una incoherencia;
3. se solicita resolución de conflicto;
4. el estado se congela en el punto observado;
5. el consenso revisa la verdad del estado.

10. Qué debe y qué no debe hacer el widget

Sí debe hacer

- mostrar permisos vigentes;
- mostrar bloqueos;
- explicar razones;
- indicar acciones alternativas;
- reflejar inconsistencias;
- servir como ventana visible de la máquina de estados.

No debe hacer

- decidir el resultado de la mano;
- modificar reglas;
- alterar el estado por sí mismo;
- crear permisos no calculados;
- sustituir al consenso;
- inventar respuestas para aparentar normalidad.

11. Valor arquitectónico

El widget convierte la lógica interna del protocolo en algo visible para el jugador.

Eso aporta:

- transparencia;
- validación humana inmediata;
- menos errores de intención;
- menor ambigüedad operacional;
- mejor trazabilidad;
- más facilidad para auditar el comportamiento de la máquina.

En Chain Poker Genesis, esto es especialmente importante porque el protocolo no depende de confianza, sino de **estados verificables**.

12. Conclusión

El Widget de Permisos de Mesa es una pieza clave de la arquitectura porque muestra de forma clara y anticipada qué puede y qué no puede hacer el jugador en un estado dado.

En un escenario de all-in, el widget puede decirle al jugador:

- si call no está permitido;
- si all-in sí está permitido;
- si raise no está permitido por falta de fondos;
- si la mesa está en inconsistencia;
- si debe solicitar resolución de conflictos.

Su función final es simple pero crítica:

convertir la máquina de estados en una guía visible, verificable y auditable para el jugador.